📄 Chuleta: permisos en Linux (LPIC-1)

> Todo en una página: imprímela, pégala en la pared… y **practica** los ejercicios de la guía.

1 · Leer `ls -l` (la receta de 5 segundos)

```
-rwxrwxr-x  1  alumno  alumno  1015  Oct  9 16:16  cerve.sh
│└┘└┘└┘   │   │       │      │     │          └ nombre
│ │  │  │   │   │       │      │     └ fecha de modificación (hora si < 6 meses, si no año)
│ │  │  │   │   │       │      └ tamaño en bytes
│ │  │  │   │   │       └ grupo propietario
│ │  │  │   │   └ usuario propietario
│ │  │  │   └ nº de enlaces (en carpetas: 2 + subcarpetas)
│ │  │  └ OTROS (o)      ┐
│ │  └ GRUPO (g)          ├ 3 trozos de 3, siempre en orden r w x; la raya = "no"
│ └ DUEÑO (u)             ┘
└ tipo: - fichero · d carpeta · l enlace · c b dispositivo · p tubería · s socket
```

2 · r, w, x: fichero frente a carpeta

	Fichero	Carpeta
`r` (4)	leer el contenido	**listar nombres** (`ls`)
`w` (2)	modificar el contenido	**crear / borrar / renombrar dentro** (necesita `x`)
`x` (1)	ejecutarlo (un **script** necesita `r` + `x`)	**entrar y acceder por nombre** (`cd` , `ls -l`)

- **Borrar** un fichero → manda la **carpeta** (`w` + `x`), no el fichero.
- Hace falta `x` en **todas** las carpetas del camino (`namei -l ruta`).

3 · ¿Quién decide? (el algoritmo)

`root` → pasa (salvo ejecutar sin ninguna `x`) · **dueño** → solo su trozo · **grupo** (principal o secundario) → solo su trozo · **otros**. **El primero que coincide gana; no se suma.** (Un dueño con `---` no puede leer, aunque "otros" sí.)

4 · Octal: r=4 · w=2 · x=1

Cifra	Binario (`r w x`)	Letras	Se suma	Significa
0	`000`	`---`	0	nada
1	`001`	`--x`	1	ejecutar
2	`010`	`-w-`	2	escribir
3	`011`	`-wx`	2 + 1	escribir + ejecutar
4	`100`	`r--`	4	leer
5	`101`	`r-x`	4 + 1	leer + ejecutar
6	`110`	`rw-`	4 + 2	leer + escribir
7	`111`	`rwx`	4 + 2 + 1	todo (leer + escribir + ejecutar)

`rwxr-xr-x` → `7·5·5` = **755** · cifras: **dueño · grupo · otros** · 4.ª cifra (si existe): **especiales** `4` =SUID `2` =SGID `1` =sticky.

5 · Los modos que debes reconocer

Octal	Letras	Para qué se usa
`000`	`---------`	nada para nadie (carpeta/fichero "cerrado")
`400`	`r--------`	solo lectura, solo el dueño (claves privadas muy protegidas)
`440`	`r--r-----`	solo lectura para dueño y grupo (`/etc/sudoers` : `-r--r-----`)
`444`	`r--r--r--`	solo lectura para todos
`600`	`rw-------`	**privado**: lee y escribe solo el dueño (clave SSH privada, `secreto.txt`)
`640`	`rw-r-----`	el dueño edita, el grupo lee (`/etc/shadow` , logs)
`644`	`rw-r--r--`	**el clásico de los ficheros**: todos leen, solo el dueño escribe (`/etc/passwd`)
`660`	`rw-rw----`	dueño y grupo editan (trabajo en equipo)
`664`	`rw-rw-r--`	como 660 pero todos leen (el que dan `umask 002` y Ubuntu)
`666`	`rw-rw-rw-`	todos leen y escriben (¡evítalo!)
`700`	`rwx------`	**privado**: script o carpeta solo del dueño (`~/.ssh`)
`711`	`rwx--x--x`	el dueño todo; los demás solo "atraviesan" (`--x`)
`750`	`rwxr-x---`	dueño todo; grupo lee y entra (**HOME en Ubuntu 24.04**)
`751`	`rwxr-x--x`	como 750 pero otros pueden ejecutar (`saludos.sh` de tu captura)
`755`	`rwxr-xr-x`	**el clásico de programas y carpetas**: todos leen/entran, solo el dueño escribe
`770`	`rwxrwx---`	dueño y grupo todo; otros nada
`775`	`rwxrwxr-x`	dueño y grupo todo; otros leen/ejecutan (`cerve.sh` y `salario.sh` de tu captura)
`777`	`rwxrwxrwx`	todo para todos (¡casi siempre un error!)
`1777`	`rwxrwxrwt`	**sticky**: carpeta pública donde cada uno borra lo suyo (`/tmp`)
`1730`	`rwx-wx--T`	buzón: el grupo deposita, nadie lista (`/var/spool/cron/crontabs`)
`2755`	`rwxr-sr-x`	**SGID** en programa: corre con el grupo del dueño (`/usr/bin/crontab`)
`2775`	`rwxrwsr-x`	**SGID** en carpeta de equipo: lo nuevo hereda el grupo
`4755`	`rwsr-xr-x`	**SUID**: corre con los privilegios del dueño (`/usr/bin/passwd`)

6 · Conversión rápida

De → A	Cómo
letras → octal	corta de 3 en 3; suma 4·r + 2·w + 1·x de cada trozo
octal → letras	cada cifra → su trozo (7 `rwx` · 6 `rw-` · 5 `r-x` · 4 `r--` · 0 `---`)
letras → binario	letra = 1, raya = 0 (`rwxr-xr-x` = `111 101 101`)
binario → octal	agrupa de 3 en 3
octal → decimal	`echo $(( 8#755 ))` = 493 · decimal → octal: `printf '%o\n' 493`

De → A	Cómo
ver el octal real	`stat -c '%a %A %n' *`
herramienta	`permisos 755` · `permisos -rwsr-xr-x` · `permisos 644 g+w` · `permisos -u 027`

7 · `chmod`

```
chmod [ugoa]  [+ - =]  [rwxXst | ugo]   f          chmod u+x,g-w,o= f        chmod 640 f        chmod -R u=rwX,go=rX d
       |       |              └─ X: x solo si es carpeta o ya era ejecutable
       |       └─ + añade · - quita · = fija EXACTAMENTE (borra el resto)
       └─ u dueño · g grupo · o otros · a todos       g=u copia los de u
```

- **Solo el dueño** (o root) hace `chmod`. `chmod -R` con números pone `x` a todos los ficheros → usa `X` o `find -type d/-type f`.
- ⚠️ **Sin "quién"** (`chmod -w f` , `chmod +x f`) **respeta la** `umask` → escribe siempre `a-w` , `go-w` …
- Un `chmod` numérico quita SUID/SGID de **ficheros**, pero **no** el SGID de **carpetas** (usa `g-s` o `00755`).
- Si **no eres del grupo** del fichero, tu `chmod g+s` **no pone** el SGID (sin avisar): usa `sudo` .

8 · `umask` : `permisos = base & ~umask` (base 666 ficheros · 777 carpetas)

`umask`	Ficheros nuevos (666 & ~umask)	Carpetas nuevas (777 & ~umask)	Uso típico
`0000`	`666` `rw-rw-rw-`	`777` `rwxrwxrwx`	nada se quita (¡peligroso!)
`0002`	`664` `rw-rw-r--`	`775` `rwxrwxr-x`	**Ubuntu** para usuarios normales (grupo escribe)
`0007`	`660` `rw-rw----`	`770` `rwxrwx---`	equipo cerrado: otros no ven nada, el grupo trabaja
`0022`	`644` `rw-r--r--`	`755` `rwxr-xr-x`	**clásica**: `root` , Debian, casi todo (grupo y otros solo leen)
`0027`	`640` `rw-r-----`	`750` `rwxr-x---`	servidores: grupo lee, otros nada
`0037`	`640` `rw-r-----`	`740` `rwxr-----`	grupo solo lee (sin `x`), otros nada
`0077`	`600` `rw-------`	`700` `rwx------`	**privado**: solo el dueño
`0222`	`444` `r--r--r--`	`555` `r-xr-xr-x`	ficheros y carpetas "de solo lectura" al nacer

Ficheros: calcula como carpeta (`7 − cifra`) y **quita la** `x` del resultado. **No es una resta** (`umask 033` → ficheros 644, no 633). Dónde: `umask` (shell) · `~/.bashrc` · `/etc/login.defs` + `pam_umask` · `UMask=` en systemd. Ubuntu: `0002` si tu grupo se llama como tú. `sudo` fuerza ≥ `022` . Obedecen la umask: `touch` , `>` , `mkdir` , `cp` . **No:** `mv` , `cp -p` , `mkdir -m` , `install -m` , `chmod` .

9 · `chown` y `chgrp`

Comando	Quién puede
`chown ana f` · `chown ana:equipo f` · `chown :equipo f` · `chown ana: f` (su grupo de login)	**solo** `root` el dueño
`chgrp equipo f`	`root` , o el dueño **si es miembro** de `equipo`
`-R` recursivo · `--from=X` · `--reference=OTRO` · `-v` `-c`	`chown` / `chgrp` **borra SUID/SGID** de un fichero: primero `chown` , luego `chmod`

`usermod -aG grupo usuario` (¡con **-a**!) y **volver a iniciar sesión** (o `newgrp` / `sg`). `sudo cp` deja ficheros de `root` → `sudo chown "$USER": f` .

10 · Especiales

Bit	Octal	Letra	En un fichero	En una carpeta
SUID	4	`s` (x del dueño)	corre con los privilegios del **dueño** (`passwd`). **Ignorado en scripts**	—
SGID	2	`s` (x del grupo)	corre con el **grupo** del fichero (`crontab`)	lo nuevo **hereda el grupo** (y el SGID)
Sticky	1	`t` (x de otros)	—	solo el dueño del fichero/carpeta o root **borra** (`/tmp` = 1777)

`S` / `T` mayúscula = el bit **sin** `x` debajo. Poner: `chmod u+s` · `g+s` · `+t` · `4755` `2775` `1777` . Auditar: `find / -perm -4000 -type f` · `-perm -2000` · `-perm /6000` (SUID **o** SGID) · `find / -xdev -type d -perm -0002 ! -perm -1000` (públicas sin sticky) · `find / -nouser -o -nogroup` . `-perm 644` exacto · `-perm -644` **todos** esos bits · `-perm /644` **alguno**.

11 · Receta de carpeta de equipo

```
sudo chgrp equipo carpeta && sudo chmod 2770 carpeta      # SGID: lo nuevo hereda el grupo
umask 002                                                 # los miembros: lo nuevo sale rw-rw----/664
```

12 · Diagnóstico de "Permission denied"

`Permission denied` = falta un **permiso** · `Operation not permitted` = acción **reservada** (chown sin root, chmod sin ser dueño, sticky, inmutable). **1** leer el mensaje · **2** `id` · **3** `namei -l ruta` · **4** `ls -ld` , `stat` · **5** `sudo diagnostica -u usuario ruta` · **6** `lsattr` , `findmnt -T` , `getfacl` , `getcap` , AppArmor · **7** arreglar con el cambio **mínimo** (nunca `777`). No es de permisos: `command not found` (PATH) · `Is a directory` · `bad interpreter` / `required file not found` (shebang o `\r`) · `Exec format error` · `Read-only file system` .

13 · Más allá de `rwx`

`chattr +i f` inmutable (ni root) · `chattr +a f` solo-añadir · `lsattr` · `getfacl` / `setfacl -m u:ana:rw f` (ACL: `+` en `ls -l`) · `setcap` / `getcap` · montajes `ro` `noexec` `nosuid` · MAC: AppArmor (Ubuntu), SELinux.

14 · Ficheros clave

`/etc/passwd` 644 · `/etc/shadow` 640 root:shadow · `/etc/sudoers` 440 · `~/.ssh` 700 y clave privada 600 · `/tmp` 1777 · `/var/mail` 2775 root:mail · `/home/usuario` 750 · `lrwxrwxrwx` (enlace: mandan los permisos del destino).